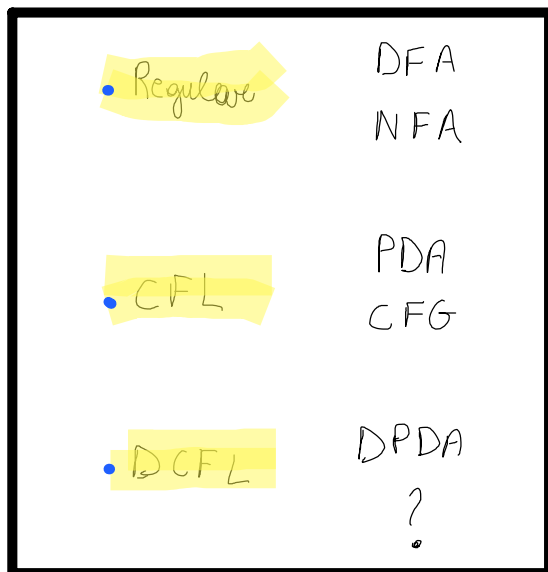




Dobbiamo definire una CFG deterministica, una DCFG!

Per arrivare a questo scopo, dobbiamo dare una definizione preliminare!

definizione: Un linguaggio è "endmarked":

- Prendiamo un linguaggio $A \in \Sigma^*$ qualunque.
- Prendiamo un simbolo che non appartiene all'alfabeto, $\rightarrow \notin \Sigma$.
- Prendiamo tutte le stringhe $w \in A$ e ne consideriamo una versione in cui mettiamo il terminatore di stringa: $w \rightarrow$

IL TERMINATORE di stringa è un modo per dire che la stringa è finita!

ALLORA, DATO UN LINGUAGGIO A , POSSIAMO DEFINIRE LA SUA VARIANTE TERMINATA:

$$A \rightarrow = \{w \rightarrow \mid w \in A\}$$

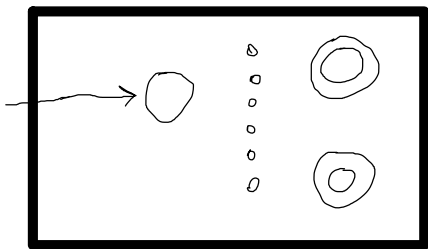
È un linguaggio a cui ho aggiunto un terminatore di stringa.

Ogni macchina che legge questo linguaggio sa perfettamente quando la stringa è finita, quando legge il carattere terminatore.

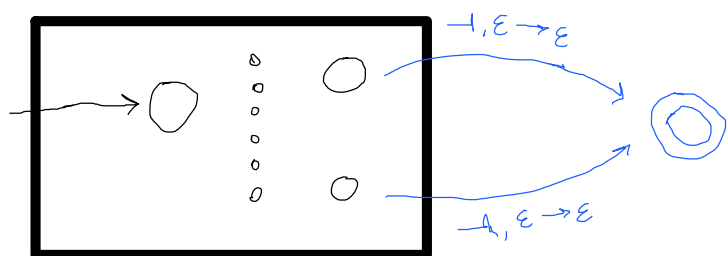
Teorema: A è DCFL se e solo se $A \rightarrow$ è DCFL.

dimostrazione: Se A è DCFL allora $A \rightarrow$ è DCFL.

Se A è un DCFL allora esiste un DPDA P : $L(P) = A$:



ORA, NOI POSSIAMO PRENDERE QUESTA MACCHINA E COSTRUIRE UNA MACCHINA IDENTICA IN CUI GLI STATI FINALI NON SONO VERAMENTE FINALI MA SONO STATI CHE PORTANO AL VERO STATO FINALE quando si legge il terminatore di stringhe.



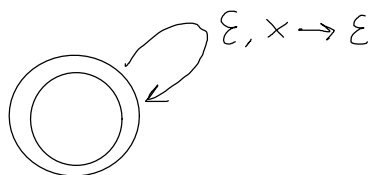
Se prima accettava, avendo terminato di leggere l'input, nell'altra macchina arriva negli stati finali non più della stringa e accetta leggendo il terminatore.

una fine una stringa e viene raggiunto il terminatore.

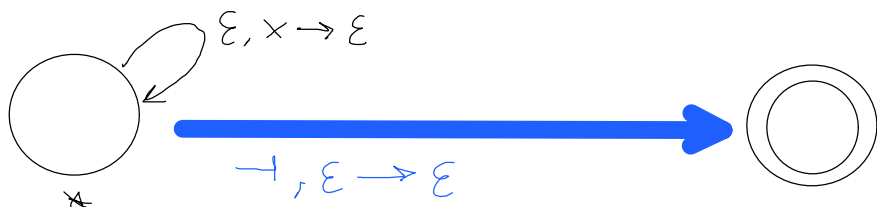
Il problema è che aggiungendo una di quelle frecce blu, potrebbe rendere l'automa "non deterministico".

Esempio

se nell'automa iniziale avevo che accettava e si mangiava tutto lo stack:



allora, quando vado a modificarlo, questo non diventa più di accettazione, poi aggiungo una nuova transizione che porta allo stato di accettazione, però l'automa non è più deterministico!



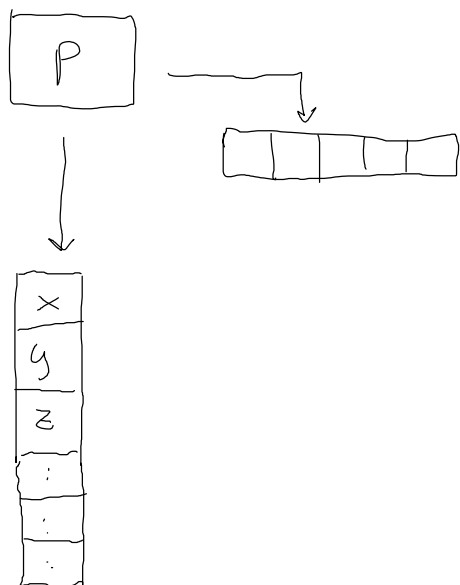
Perché se lo stack non è vuoto, quando sono qui (*) posso scegliere di leggere il carattere terminatore oppure leggere dallo stack.

Si può risolvere

dimostrazione: se $A \rightarrow$ è un DCFL allora A è DCFL.

questa è la parte difficile perché qui bisogna dimostrare che, anche se ci tolgo il terminatore finale continua ad essere riconoscibile dalla macchina deterministica.

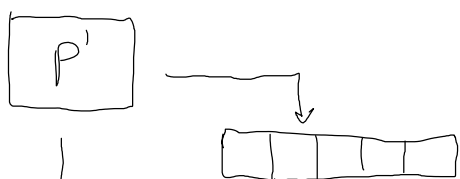
supponiamo di avere:



questa macchina riconosce il nostro linguaggio $A \rightarrow$.

$$L(P) = A \rightarrow$$

L'idea è che bisogna costruire una macchina P' che simula:



Lo stack viene usato per due scopi:

- PER simulare lo stack della prima macchina
- PER TENERSI TRACCIA di alcune informazioni.



Sopra ogni carattere sullo stack, ci mettiamo anche altre informazioni!

Il mio scopo è fare in modo che questa macchina riconosca A , senza il terminatore finale.

$$L(P') = A$$

$$P = (Q, \Sigma \cup \{\rightarrow\}, \Gamma, \delta, q_0, F)$$

$$P' = (Q', \Sigma, \Gamma', \delta', q'_0, F') \quad \Gamma' = \Gamma \cup \Sigma^Q$$

che cos'è R_x ?

codifica gli stati di P (della macchina originale) tale che P può entrare in uno stato di accettazione senza leggere nessun simbolo dell'input.

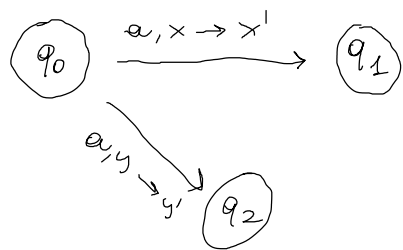
cioè R_x rappresenta tutti gli stati che, sulla base della configurazione $x^i y^j z$ dello stack, possono portare P ad entrare in uno stato di accettazione senza leggere nessun simbolo dell'input.

Quindi ciascuno di questi R non è altro che un sottoinsieme di stati: $R \subseteq Q$.

Noi possiamo sempre ricondurre le transizioni della macchina P che sono:

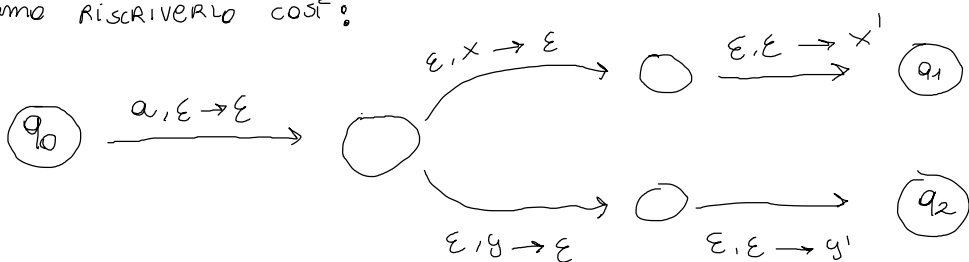
$$\delta : \begin{cases} \text{Lettura} & \delta(q, a) = (r, \epsilon) \\ \text{Push} & \delta(q, \epsilon, x) = (r, x) \\ \text{Pop} & \delta(q, \epsilon, x) = (r, \epsilon) \end{cases}$$

Non è difficile vedere che, se io ho uno stato con delle transizioni generiche,



È sempre possibile costruire una macchina deterministica che fa un passo alla volta.
cioè:

possiamo riscriverlo così:

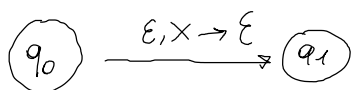


E questa è deterministica!

Questo per la macchina P .

Come faccio a trasformare dalla macchina P alla macchina P' ?
POP

Ora vediamo la macchina P :



La macchina P' , per fare la pop, deve fare la pop di due caratteri, cioè se poppo x da P , devo poppare x ed R_x da P' .

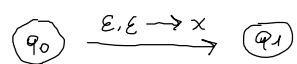


Prima faccio la pop di R

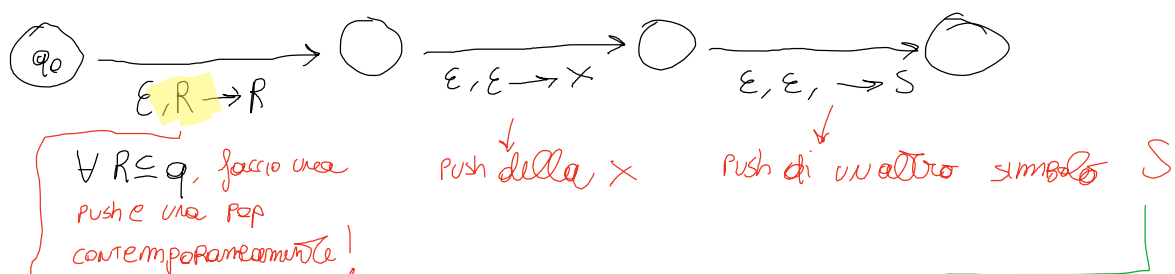
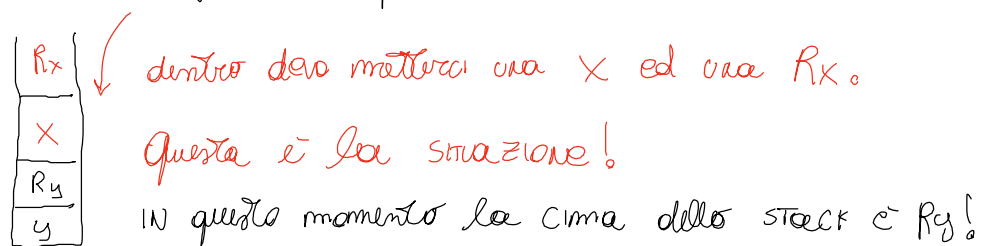
ora faccio la pop della x

PUSH

Se P fa una push:



allora P' deve fare due push:



$\rightarrow S$ è l'insieme di tutti gli stati d'accettazione, perché se la mia macchina è in uno stato di accettazione ovviamente accetta senza dover esaminare lo stack!

Questi sono tutti gli stati che portano la macchina ad accettare senza dover leggere nulla dell'input, OK perfetto, se sono in uno stato di accettazione accetto, punto.

IN PIÙ ci devo mettere tutti gli stati, tali che, se da questo stato faccio una pop senza leggere l'input della x , allora arrivo in uno stato r che $\in$ allo stato di cui ho fatto la pop.

$\rightarrow$ qui ho appena messo gli stati sulla cima dello stack.

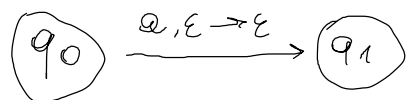
Quando simulo una push, prima mi leggo R e me lo salvo negli stati, faccio la push di x e poi dico:

"va bene ma il nuovo S sono tutti gli stati che, quando faccio la pop di x mi portano in uno stato che sta in R_y (lo stato sotto nello stack) e quindi ho costruito le nuove stati da mettere sulla cima dello stack.

$$S = F \cup \{q \in Q \mid \delta(q, \epsilon, x) = (r, \epsilon), r \in R\}$$

Read

Se nella macchina P leggo il carattere a e vado in q_1 :



allora in P' :

(duplico tutti gli stati in una versione accettante/non accettante)

